

```
In [1]: from scipy.optimize import linprog
```

1b.

```
In [2]: # max z == min -z; taking negative of coeff to maximize
c = [3, -4, -1, -2, -3]

# Constraints
A = [[1, 1, -1, -1, 2], [-1, 2, 1, 1, 1]]

# RHS
b = [12, 30]

# x1,...,x5 >= 0
x_bounds = [(0, None), (0, None), (0, None), (0, None), (0, None)]

# simplex
result = linprog(c = c, A_ub = A, b_ub = b, bounds = x_bounds, method = 'highs')

if result.success:
    print("Value (max z):", -result.fun) # using - to maximize, linprog minimizes d
    print("Solution (x1, x2, x3, x4, x5):", result.x)
else:
    print("No solution")
```

Value (max z): 74.0

Solution (x1, x2, x3, x4, x5): [0. 0. 0. 16. 14.]

2b.

```
In [3]: # max z == min -z
c = [-2, -2]

A = [[0, 1],
     [1, -3],
     [1, 1]]

b = [10, 2, 16]

x_bounds = [(0, None), (0, None)]

# simplex
result = linprog(c, A_ub = A, b_ub = b, bounds = x_bounds, method = 'highs')

if result.status == 3:
    print("Unbounded!")
elif result.success:
    print("(max z):", -result.fun) # using - to maximize
    print("(x1, x2):", result.x)
```

```
else:  
    print("No solution")
```

(max z): 32.0

(x1, x2): [6. 10.]

3b.

```
In [4]: c = [-4, -1]  
A = [[2, -3], [-4, 1]]  
b = [12, 8]  
x_bounds = [(0, None), (0, None)]  
  
# simplex  
result = linprog(c, A_ub = A, b_ub = b, bounds = x_bounds, method = 'highs')  
  
if result.status == 3:  
    print("Unbounded!")  
elif result.success:  
    print("(max z):", -result.fun) # using - to maximize  
    print("(x1, x2):", result.x)  
else:  
    print("No solution")
```

Unbounded!

In []:

In []: